Course Mini Project: Personal Reflection & Learning Log

CS3081: Cloud Computing — Knowledge Distillation & Edge Microservice Cloud Deployment

Name: Niraj Kumar **Roll No:** 230101037 **GitHub:** https://github.com/NirajS2118/cloud-assignment **Institute:** IIIT Senapati, Manipur

1. Overview and Core Learning Journey

This project was an exceptionally enriching, multidisciplinary capstone bridging Modern Cloud Native Engineering and Natural Language Processing. Prior to this assignment, my exposure to Large Language Models (LLMs) was predominantly user-facing or API-centric. Through this comprehensive project, I transitioned from conceptual understanding to hands-on systems engineering, deploying a production-style, containerized AI inference microservice orchestrated with Kubernetes on constrained local resources.

The central learning objective was understanding how sequence-level knowledge distillation enables tiny models (such as EleutherAI's 70-million-parameter Pythia-70M) to emulate complex reasoning patterns synthesized by large models (Phi-3.5-mini at 3.8 billion parameters), producing production-grade structured JSON voice parser outputs while keeping computational footprint minimal.

2. Knowledge Distillation: Conceptual Insights & Teacher-Student Dynamics

Exploring distillation hands-on dispelled several theoretical misconceptions. I realized that *logit-based distillation* is not a universal panacea: in real-world heterogeneous architectures, mismatched tokenizers (Phi-3.5's 32K SentencePiece vs Pythia-70M's 50K byte-level BPE) and black-box inference servers prevent direct probability alignment.

Consequently, *sequence-level distillation* (Kim and Rush, 2016) provided the pragmatic breakthrough: treating the teacher's greedily decoded JSON sequence as ground-truth targets. The empirical impact was startling: the base untrained Pythia-70M scored exactly **0.0% JSON validity**, outputting unstructured babble. After fine-tuning on 1,500 teacher-distilled pairs for just a single epoch, the student's JSON structural validity jumped to **99.0%**, and its full-frame semantic accuracy rose from 0% to **16.0%**. This concretely demonstrated how knowledge distillation compresses syntactic constraints into tiny neural networks.

3. Object Storage and Scalable Data Sharding Architecture

Implementing SeaweedFS as an in-cluster S3 object store revealed fundamental cloud architecture principles:

- **Decoupling Compute and Storage:** Storing datasets and model artifacts in SeaweedFS rather than baking weights into Docker image layers allowed our container images to remain lean (1.87 GB), rapid to spin up, and reusable across model iterations.
- **Parallel Worker Sharding:** Generating 3,000 teacher inferences on local CPU hardware represented an immense computational hurdle (~2–3 hours). By implementing an index-modulo worker partitioning scheme (`line_number % SHARD_COUNT == SHARD_INDEX`) across concurrent Kubernetes batch jobs, I observed linear throughput scaling across CPU cores without duplicate processing.
- **Deterministic Dataset Auditing:** Building `worker/clean.py` taught me the rigor of data engineering. Rigorously filtering invalid JSON (75 records), malformed keys (4), invalid intents (82), non-substring slot hallucinations (359), and duplicates (45) yielded 2,435 verified clean records whose SHA-256 digest guaranteed reproducibility across environments.

4. Containerization & Edge Microservice Optimization

Building the inference container highlighted the critical distinction between development and production images. Default PyTorch distributions bundle gigabytes of CUDA and cuDNN binaries, ballooning images beyond 5–7 GB. By utilizing multi-stage builds and pinning CPU-only wheels (`--extra-index-url https://download.pytorch.org/whl/cpu`), I compressed the service container to **1.87 GB**, well within the 2 GB assignment limit.

Furthermore, rigorous memory profiling under strict cgroup limits (1 CPU core, 1 GiB memory) demonstrated that our FastAPI service, even under continuous inference bursts across 50 sequential requests, maintained a steady peak resident memory of **493 MiB** (48.1% of ceiling), guaranteeing immunity from Out-Of-Memory (OOM) kernel kills.

5. Kubernetes Orchestration, Probes, Scaling, and High Availability

Deploying the application within a local `k3d` cluster deepened my appreciation of Kubernetes as a self-healing control plane:

- **The Critical Role of Probes:** Without properly configured readiness probes (`/readyz`), Kubernetes immediately routes incoming ingress traffic to newly created pods before PyTorch completes weight loading from S3. By orchestrating startup, liveness (`/healthz`), and readiness probes, our service maintained zero dropped connections during pod initialization.
- **Zero-Downtime Rolling Updates & Rollbacks:** Testing a live deployment rolling update from `v1` to `v2` while maintaining active Locust traffic demonstrated Kubernetes' rolling deployment strategy. Traffic seamlessly transitioned across replicas with a **0.0% error rate**, and executing `kubectl rollout undo` restored `v1` instantaneously.
- **Self-Healing Recovery:** When I deliberately killed a running pod during active load testing, Kubernetes automatically detected the replica deficit and scheduled a replacement pod that downloaded weights, verified checksums, and restored the replica count to 2, all with **0.0% dropped requests**.
- **Horizontal Pod Autoscaler (HPA):** Configuring metrics-server CPU-based autoscaling (min 1, max 2, 70% target utilization) illustrated how cloud elasticity dynamically absorbs traffic spikes while preventing resource overprovisioning.

6. Software-Defined Networking: Stateless NACLs vs Stateful Firewalls

Working through Task 5 provided profound clarity on networking layers and Access Control Lists. Designing the public subnet NACL (`network/nacl.json`) required adopting a strict packet-by-packet mindset:

> **Key Networking Insight:** In stateful firewalls, established connections automatically allow return packets. In contrast, a *stateless NACL* has no connection memory; outbound responses destined for client ephemeral ports (1024–65535) or internal app subnet replies must be explicitly permitted, or the traffic is blacklisted.

Additionally, implementing first-match-wins rule ordering demonstrated why security blacklists must always precede broad allow rules: placing the malicious range `198.51.100.0/24` at Rule 90 ensured complete denial before broad web allow rules (Rules 100/110) were evaluated. Simulating the rules yielded a perfect **100.0% pass rate** (6/6 sample packets).

7. Engineering Challenges Faced & Technical Solutions

- **Thread Contention on Multi-Core Laptop CPU:** During early student training runs, PyTorch CPU training stalled due to OpenMP thread lock contention across hyperthreaded laptop cores. *Resolution:* Explicitly invoking `torch.set_num_threads(4)` and pinning DataLoader batch sizes eliminated lock overhead, reducing step latency significantly.
- **Readiness Race Condition During Cold Starts:** Pods initially reported ready before model weights were downloaded from SeaweedFS, leading to HTTP 500 errors on early requests. *Resolution:* Rewrote the FastAPI application using asynchronous `lifespan` context managers and gated `/readyz` strictly on `READY = True` after cryptographic SHA-256 verification.
- **Hallucinated Substring Slot Values:** Because the student model is compact, it occasionally hallucinated extra words or rephrased dates (e.g. "two pm" instead of "2 pm"). *Resolution:* Implemented two-tiered runtime validation in `service/app.py` that enforces exact substring matching and graceful HTTP 500 error propagation when output schemas are breached.

8. Key Empirical Findings & Quantitative Observations

- **Inference Latency:** Across Locust load testing (4 users, 300+ requests), the service achieved a median latency of **740 ms** and a p95 of **900 ms**, well beneath the 5,000 ms SLA boundary.
- **Throughput & Scaling:** The 2-replica deployment sustained **5.13 requests per second** (~18,468 requests/hour) with Traefik Ingress balancing traffic almost perfectly (56.15% vs 43.85% distribution across pods).
- **Cold-Start Metric:** Measured three cold-start iterations from container creation to Ready status: 10.72s, 10.73s, and 10.76s, yielding a **median cold start of 10.73 seconds**.
- **Economic Efficiency:** Utilizing `rate_card.json`, running two replicas at 1 vCPU and 1 GiB limits costs ₹7.90/hour. At 18,468 requests/hour, this translates to an extraordinarily cost-effective **₹0.43 per 1,000 requests**.

9. What Could Be Done Better & Future Roadmap

Reflecting on the architecture, several meaningful improvements could be pursued in a production cloud setting:

- *Model Architecture:* Replacing Pythia-70M with a modern quantized 1-billion parameter Small Language Model (e.g. Llama-3.2-1B-Instruct or Qwen-2.5-0.5B) would substantially boost full-frame accuracy beyond 16% without exceeding a 1 GiB memory footprint.
- *Inference Acceleration:* Exporting PyTorch weights to **ONNX Runtime** with INT8 quantization or running via `llama.cpp` / GGML would reduce inference latency from 740 ms to under 120 ms and shrink memory consumption to <100 MiB.
- *Serverless Scale-to-Zero:* Implementing KEDA or Knative would eliminate the idle ₹7.90/hour server cost by scaling replicas to zero during off-peak periods.

10. Concluding Thoughts

This assignment delivered invaluable end-to-end practical mastery across distributed cloud systems. Developing reproducible workflows using `make`, automating multi-stage CI/CD pipelines with quality gates in GitHub Actions, and designing fault-tolerant Kubernetes deployments reinforced the principles of robust software engineering. I now feel thoroughly equipped to architect, containerize, and deploy intelligent microservices on modern cloud infrastructure.